U.D. 8. PL-SQL.

CENTRO FP SUPERIOR
CAMPUS CÁMARA COMERCIO SEVILLA

BBDD

Nuria Fuentes Pérez

INDICE

1. Introducción a PL-SQL.
2. Bloque anónimo.
3. Sintaxis PL-SQL.
   A. Indentificadores variables y constantes.
   B. Operadores.
   C. Instrucciones.
      a. Asignación.
      b. Alternativas.
      c. Repetitivas.
   D. Interacción con el usuario.
   E. Otros.
      a. Funciones del SGBD.
      b. Cursores.

INDICE

4. Programas almacenados.
   A. Procedimientos.
   B. Funciones.
   C. Subprogramas locales.
   D. Recursividad.

1. Introducción a PL-SQL.

•PL/SQL significa Procedural Language/SQL
•PL/SQL es un sofisticado lenguaje de programación de cuarta generación que se utiliza para acceder a bases de datos Oracle desde distintos entornos.
•Los programas creados en PL/SQL se almacenan en la BD como cualquier otro objeto.
•PL/SQL está integrado con el servidor de base de datos
•El PL/SQL es un lenguaje como el SQL y no un producto
Sus principales características son:

- Lenguaje de programación. (administradores y desarrolladores)
- Una extensión procedural del SQL.
- Combinación del:
  - ✓ Poder de manipulación de datos del SQL.
  - ✓ Poder de procesamiento de los lenguajes procedurales.

1. Introducción a PL-SQL.

En PL/SQL hay 3 tipos de bloques que trabajan con el motor del servidor:

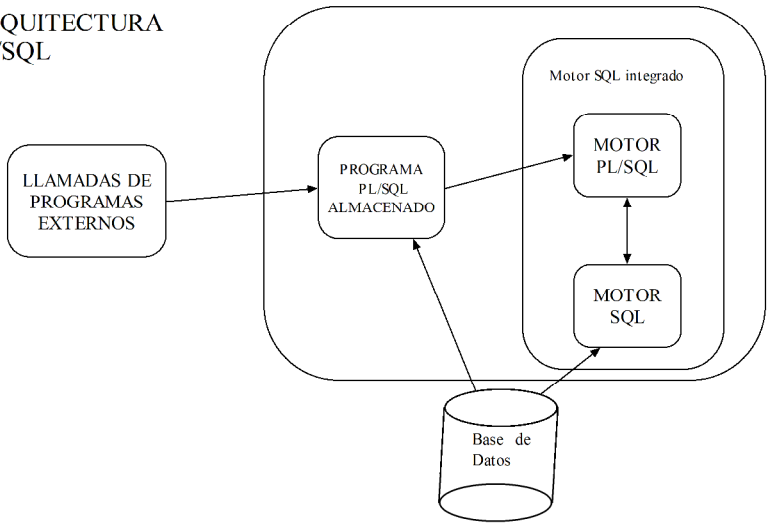
- **Bloque anónimo**: se envían al servidor dónde serán compilados y ejecutados.

- **Subprograma almacenado**: procedimientos y funciones que se compilan y almacenan en la BD. Estos se pueden agrupar en paquetes que también se almacenan en la BD, llamándose entonces subprogramas empaquetados.

- **Disparadores de la BD**: subprogramas almacenados que se asocian a una tabla de la BD y se ejecutan automáticamente cuando se producen determinados cambios en las mismas.

1. Introducción a PL-SQL.

ARQUITECTURA PL/SQL



2. Bloque anónimo.

Las sentencias PL/SQL se agrupan formando una estructura básica llamada **Bloque**.
Las sentencias son básicamente de dos tipos: SQL y NO-SQL
Los bloques PL/SQL se escriben dentro del SQL Plus, SQL Plus Worksheet o de otra herramienta Oracle (por ejemplo SQL*Forms).
La estructura general de un bloque tiene 3 zonas diferenciadas: declaraciones, instrucciones y excepciones

```
[DECLARE
   <zona de declaraciones de variables>]
BEGIN
   <sentencias ejecutables>
 [EXCEPTION
    <sentencias de control de errores y excepciones>]
END;
```

3. Sintaxis PL-SQL.
A. Indentificadores variables y constantes.

COMENTARIOS

-- línea comentada

/*
párrafo comentado
*/

IDENTIFICADOR

- 30 caracteres máximo.
- Empieza por letra, seguido de (A-Z, 0-9, $, #, _)
- No distingue entre mayúsculas y minúsculas.

VARIABLES

Se deben declarar en la sección DECLARE, y se utilizan para:

· Almacenar datos devueltos por una consulta.
· Almacenar cálculos intermedios.

Se declaran:

<nVariable> <tipoDato> [**NOT NULL**] [{:= | **DEFAULT**}<expresión PL/SQL> | valor];

Para cada variable su tipo, **NO** se puede especificar una lista.

3. Sintaxis PL-SQL.
A. Indentificadores variables y constantes.

CONSTANTES

<nCte> **CONSTANT** <tipo> := <valor>;

TIPOS DE DATOS

BINARY_INTEGER **NUMBER** LONG RAW
DEC NUMERIC RAW
DECIMAL POSITIVE ROWID
DOUBLE PRECISION REAL STRING
FLOAT SMALLINT VARCHAR
DATE INT **VARCHAR2**
INTEGER CHARACTER **CHAR**
NATURAL LONG **BOOLEAN**

TABLE **RECORD**

3. Sintaxis PL-SQL.
A. Indentificadores variables y constantes.

```
importe        NUMBER(8,2);
provincia      VARCHAR2(20) DEFAULT 'SEVILLA';
valor          NUMBER(4)    DEFAULT      0;
valor2         NUMBER(4)    :=0;
POB            CONSTANT CHAR(7):='SEVILLA';
```

Para cada variable su tipo, **NO** se puede especificar una lista.

%TYPE ➔ coger el tipo de otra variable o campo.

```
numeroDept     DEPART.DEPT_NO%TYPE;
```

%ROWTYPE ➔

```
fila           EMPLE%rowtype;
```

Se hereda el tipo pero no se heredan los valores por defecto ni los atributos NOT NULL.

3. Sintaxis PL-SQL.
B. Operadores.

LÓGICOS: AND OR NOT (perezosos)

CONCATENACIÓN: ||

RELACIONALES: = != < > <= >=
IS NULL BETWEEN
LIKE IN

ARITMÉTICOS: + - * / **

fec1-fec2
fec+n
fec-n

+
** NOT
* /
+ - ||
= != < > <= >= IS NULL
BETWEEN LIKE IN
AND
OR
-

3. Sintaxis PL-SQL.
C. Instrucciones.

ASIGNACIÓN

<nVariable>:=<expresión>;

```
importe := 1000;
valor := importe * 0.15;
```

ALTERNATIVA

IF <condición> **THEN**

<sentencia PL/SQL>;

[**ELSEIF** <condición> **THEN**

<sentencia PL/SQL>;]

...

[**ELSE**

<sentencia PL/SQL>;]

END IF;

3. Sintaxis PL-SQL.
C. Instrucciones.

REPETITIVA

Bucle ITERAR.

LOOP
  <secuencia de sentencias>
 EXIT WHEN <condición>;
  <secuencia de sentencias>
END LOOP;

Bucle WHILE

WHILE <condición> **LOOP**
  <secuencia de sentencias>
END LOOP;

Bucle FOR

FOR <variable> **IN** [**REVERSE**] <mínimo>..<máximo> **LOOP**
  <secuencia de sentencias>
END LOOP;

3. Sintaxis PL-SQL.
D. Interacción con el usuario.

PL/SQL: no posee instrucciones para interactuar con el usuario.

Posee un paquete para depurar código DBMS_OUTPUT que posee un procedimiento para visualizar datos en pantalla:

DBMS_OUTPUT.PUT_LINE(<expresión>);

Para que funcione correctamente deberemos habilitar la salida en la consola activando la variable de entorno:

SET SERVEROUTPUT ON;

Entradas:

- Variables de sustitución de SQl*Plus.
- Parámetros en procedimientos y funciones.

3. Sintaxis PL-SQL.
D. Interacción con el usuario.

```
BEGIN
        DBMS_OUTPUT.PUT_LINE('Hola mundo');
END;
```

No olvides habilitar
la salida por pantalla.

3. Sintaxis PL-SQL.
E. Otros.

FUNCIONES

Se pueden usar las funciones de SQL. Las funciones de agrupamiento sólo se podrán utilizar en las sentencias SELECT.

CURSORES

Un cursor sirve para guardar el resultado de una consulta.

El resultado de cualquier instrucción SQL va a un área de memoria a la que accede el cursor.

Una sentencia **SELECT** puede devolver cero, una o más de una fila y eso hace que distingamos 2 tipos de cursores:

- Cursores implícitos
  sentencias que devuelven una sola fila
- Cursores explícitos
  sentencias que devuelven más de una fila

3. Sintaxis PL-SQL.
Cursores implícitos.

Consultas que devuelvan una sola fila guardando el resultado en variables

SELECT columna/s **INTO** variable/s **FROM** tabla/s;

```
DECLARE
  dept DEPART.DEPT_NO%TYPE;
  total NUMBER(10);
BEGIN
   SELECT DEPT_NO, SUM(SALARIO) INTO dept, total
            FROM EMPLE
            WHERE DEPT_NO=20
            GROUP BY DEPT_NO;
   DBMS_OUTPUT.PUT_LINE(dept||': '||total);
END;
```

3. Sintaxis PL-SQL.
EJEMPLOS

HACER LOS SIGUIENTES BLOQUES:

- Mostrar "hola mundo"
- Mostrar la suma de los 100 primeros naturales usando todas las repetitivas, indicando si el resultado es par o impar
  La suma es XXXX y es XXXXX
- Mostrar el nombre del departamento 10.
- Mostrar la localidad del departamento solicitado.

4. Programas almacenados.

Los subprogramas son bloques PL/SQL identificados por un nombre, que pueden recibir parámetros y ser invocadas. PL/SQL posee dos tipos de subprogramas:

- Procedimientos.
- Funciones.

La estructura de un subprograma es:

La cabecera contiene la especificación del subprograma, nombre parámetros, tipos, etc..

El cuerpo del subprograma con la misma estructura que un bloque "anónimo"

Una zona declarativa: Contiene variables, constantes, excepciones y programas enlazados. Estos objetos son locales y dejan de existir cuando salimos de subprograma.

Una zona ejecutable: Contiene instrucciones, asignaciones de variables, instrucciones SQL.

Una zona de control de errores o excepciones: Contiene instrucciones de control de excepciones.

4. Programas almacenados.
A. Procedimientos.

```
PROCEDURE nombre [ (listaDeParámetros) ]
IS
   declaraciones locales
BEGIN
   instrucciones
[EXCEPTION
   control de errores]
END [nombre];
```

listaDeParámetros: param1,param2,...

- No se especificará tamaño en los parámetros.
- Si no hay parámetros no se pondrán paréntesis.
- Debe tener mínimo una línea de código.

4. Programas almacenados.
A. Procedimientos.

<nParámetro> [**IN**|**OUT**|**IN OUT**] <tipoDato> [{:=|**DEFAULT**}<valor>]

Tipo	Característica
IN	o parámetro entrada al subprograma. o en el subprograma actúa como una cte. o parámetro actual puede ser cte, var, expr,literal
OUT	o parámctro salida dcl subprograma. o en el subprograma actúa como var sin inicializar o parámetro actual debe ser var.
IN OUT	o parámetro de entrada y salida. o en el subprograma actúa como una var inicializada o parámetro actual debe ser una var.

4. Programas almacenados.
A. Procedimientos.

Llamada

```
EXECUTE nProcedimiento(<parámetrosActuales>);
```

El paso de parámetros se puede hacer usando tres notaciones:

- Notación posicional: el compilador asocia los parámetros actuales con los formales basándose en la posición.

- Notación nominal: el símbolo **=>** después del parámetro actual y antes del formal indica al compilador la correspondencia.

- Notación mixta: usar ambas con la restricción de que la posicional debe preceder a la nominal.

```
PROCEDURE gestionDept(nDept NUMBER,
                      localidad VARCHAR2)
IS...
```

```
gestionDept(numDep,local);
gestionDept(local => localidad, numDep =>nDept);
gestionDept(numDep, local => localidad)
```

4. Programas almacenados.
A. Procedimientos.

```
CREATE OR REPLACE
  PROCEDURE multiplicaNumeros (
        num1 IN NUMBER,
        num2 IN NUMBER)
 IS
   producto NUMBER(6);
 BEGIN
        producto := num1 * num2;
        DBMS_OUTPUT.PUT_LINE('El producto es: '|| producto);
 END multiplicaNumeros;
```

```
SHOW ERRORS
```

EJECUCION

```
SET SERVEROUTPUT ON;
EXECUTE multiplicaNumeros(13,26);
```

VISTAS: USER_OBJECTS
USER_SOURCE

```
SELECT LINE,SUBSTR(TEXT,1,60)
   FROM USER_SOURCE
   WHERE NAME='multiplicaNumeros';
```

4. Programas almacenados. EJEMPLOS

- Hacer un procedimiento **verEmpleado** que muestre el empleado solicitado y la localidad en la que trabaja:
El empleado XXXXXX trabaja en XXXXXX
- Hacer un procedimiento de nombre **cambiaOficio** que cambie al oficio proporcionado el oficio del empleado también proporcionado, devolviendo el oficio actual.
- Hacer un procedimiento que ejecute el procedimiento anterior mostrando el mensaje

"9999 *Oficio anterior: XXXXXXX * Oficio nuevo: XXXXX"

4. Programas almacenados.
B. Funciones.

Una función es un subprograma que devuelve un valor.
Las funciones y procedimientos tienen la misma estructura con la única diferencia de que la función posee una cláusula **RETURN** que indica el tipo de valor que devuelve.
La sintaxis de una función es:

```
FUNCTION nombre [ (parámetro [, parámetro, ...]) ]   RETURN tipo_dato
 IS
     declaraciones locales
   BEGIN
     instrucciones
     RETURN <expresión>;
   [EXCEPTION
     control de errores]
   END [nombre];
```

Una función se llama como parte de una expresión

```
IF cadenaReves(frase)=frase  THEN
   DBMS_OUTPUT.PUT_LINE( 'PALINDROMO');
END IF;
```

4. Programas almacenados.
EJEMPLO

```
CREATE OR REPLACE
  FUNCTION fMultiplicaNumeros (
        num1 IN NUMBER,
        num2 IN NUMBER) RETURN NUMBER
 IS
   producto NUMBER(6);
 BEGIN
        producto := num1 * num2;
        RETURN producto;
 END fMultiplicaNumeros;
```

4. Programas almacenados. EJEMPLOS

- Hacer una función **fVerEmpleado** que devuelva la localidad en la que trabaja el empleado solicitado:
- Hacer una función de nombre **fCambiaOficio** que cambie al oficio proporcionado el oficio del empleado también proporcionado, devolviendo el oficio actual.
- Hacer un procedimiento que ejecute la función anterior mostrando el mensaje

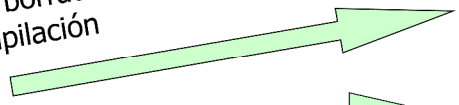
  "9999 *Oficio anterior: XXXXXXX * Oficio nuevo: XXXXX"

4. Programas almacenados.

CREATE PROCEDURE
CREATE FUNCTION

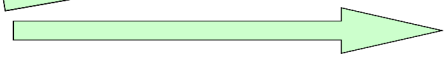
P-CÓDIGO (código objeto)
Quedan disponibles para su uso

Si alguno de los objetos referenciados ha sido borrado o modificado desde la última compilación



No disponible (INVALID)

P-CÓDIGO



Disponible (VALID)

VISTAS: USER_OBJECTS
USER_SOURCE

PARA VOLVER A COMPILAR

ALTER {PROCEDURE|FUNCTION} nPrograma **COMPILE**;

4. Programas almacenados.
C. Subprogramas locales.

```
CREATE OR REPLACE
  PROCEDURE P1
   ...
  IS
   ....
  PROCEDURE P2
    ....
   IS
     ....
   BEGIN
     ....
   END P2;
  BEGIN
      .../* llamada al P2*/
  END P1;
```

- Se declara al final de la sección declarativa
- Tienen las mismas reglas de ámbito y visibilidad que las variables
- Se usan en subprogramas que no se vayan a utilizar fuera del programa
- Si hay referencias cruzadas, habrá que declararlo antes.(cabecera)

4. Programas almacenados.
D. Recursividad.

```
CREATE OR REPLACE
  FUNCTION factorial (n NUMBER) RETURN NUMBER
  IS
    fac NUMBER(10);
  BEGIN
      IF n=1 THEN
          fac:= 1;
      ELSE
          fac:= n*factorial(n-1);
      END IF;
      RETURN fac;
  END factorial;

CREATE OR REPLACE
  PROCEDURE pruebaFactorial (n POSITIVE)
  IS
  BEGIN
      DBMS_OUTPUT.PUT_LINE(n||'!= '||factorial(n));
  END pruebaFactorial;
```

NO ENTRA